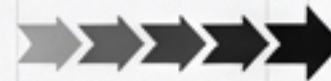


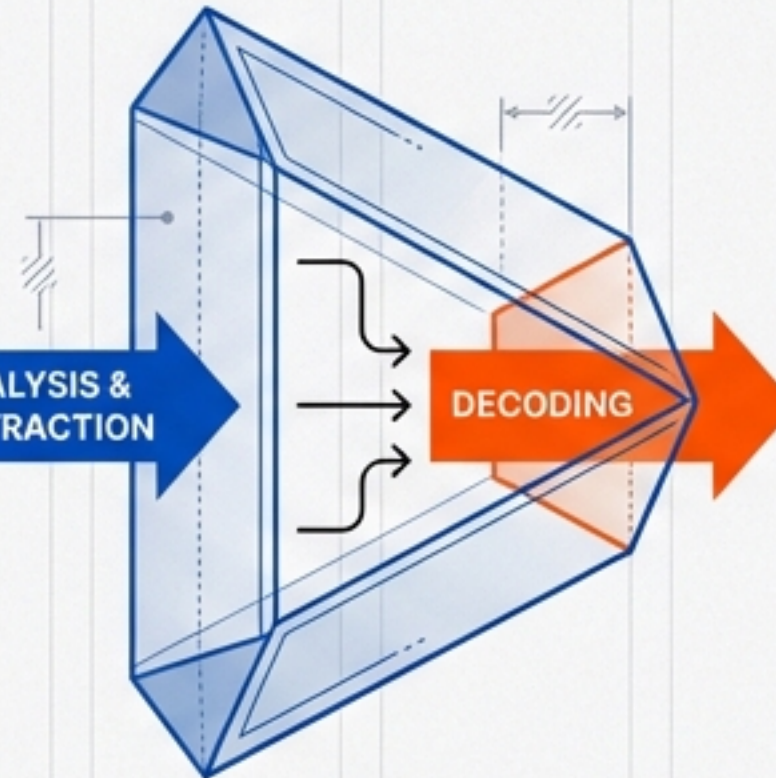
Time Complexity: The Pattern Recognition Handbook

A practical guide to decoding algorithm efficiency by analyzing code shape.

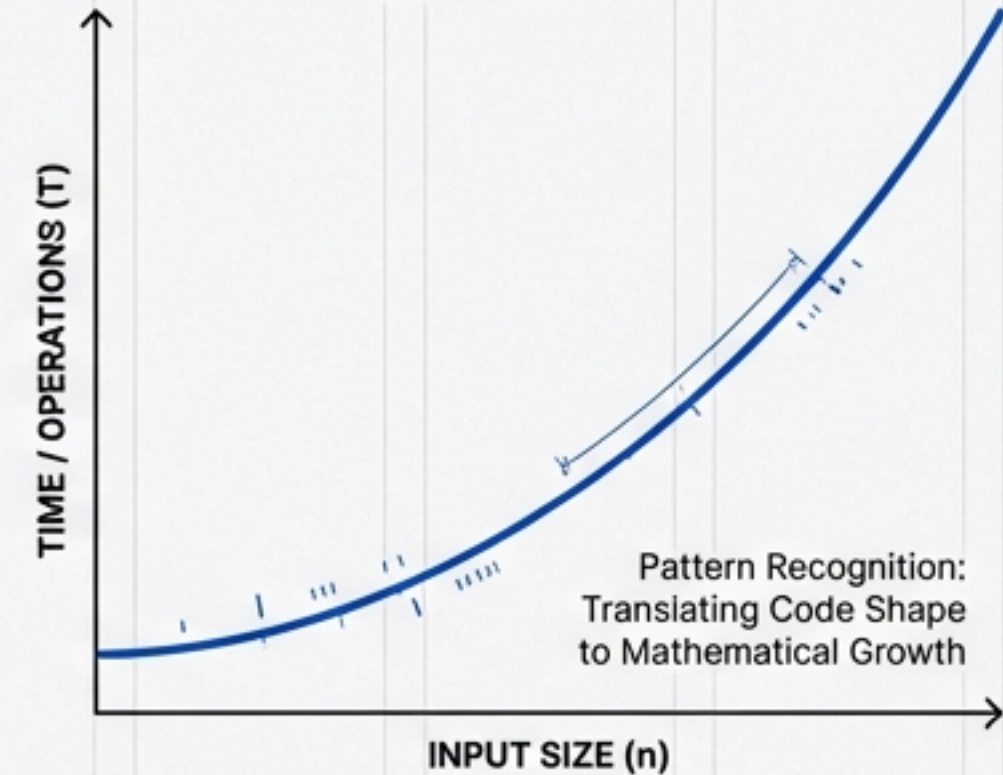
INPUT: CODE STRUCTURE →



ANALYSIS & ABSTRACTION



OUTPUT: MATHEMATICAL EFFICIENCY MODEL
($O(n)$, $O(n^2)$, etc.) →

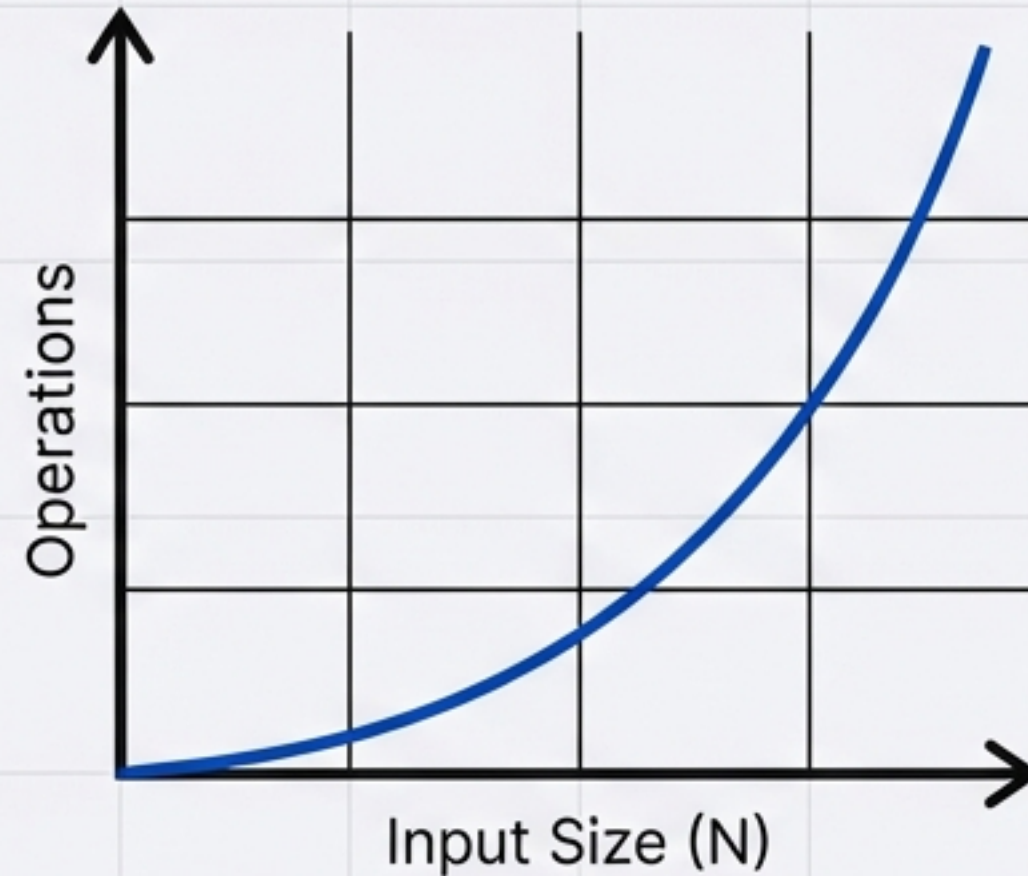


Based on the Master Rules of Big O Notation

Complexity is Behavior, Not Clock Time



Dependent on Hardware
(Mac vs. Windows)

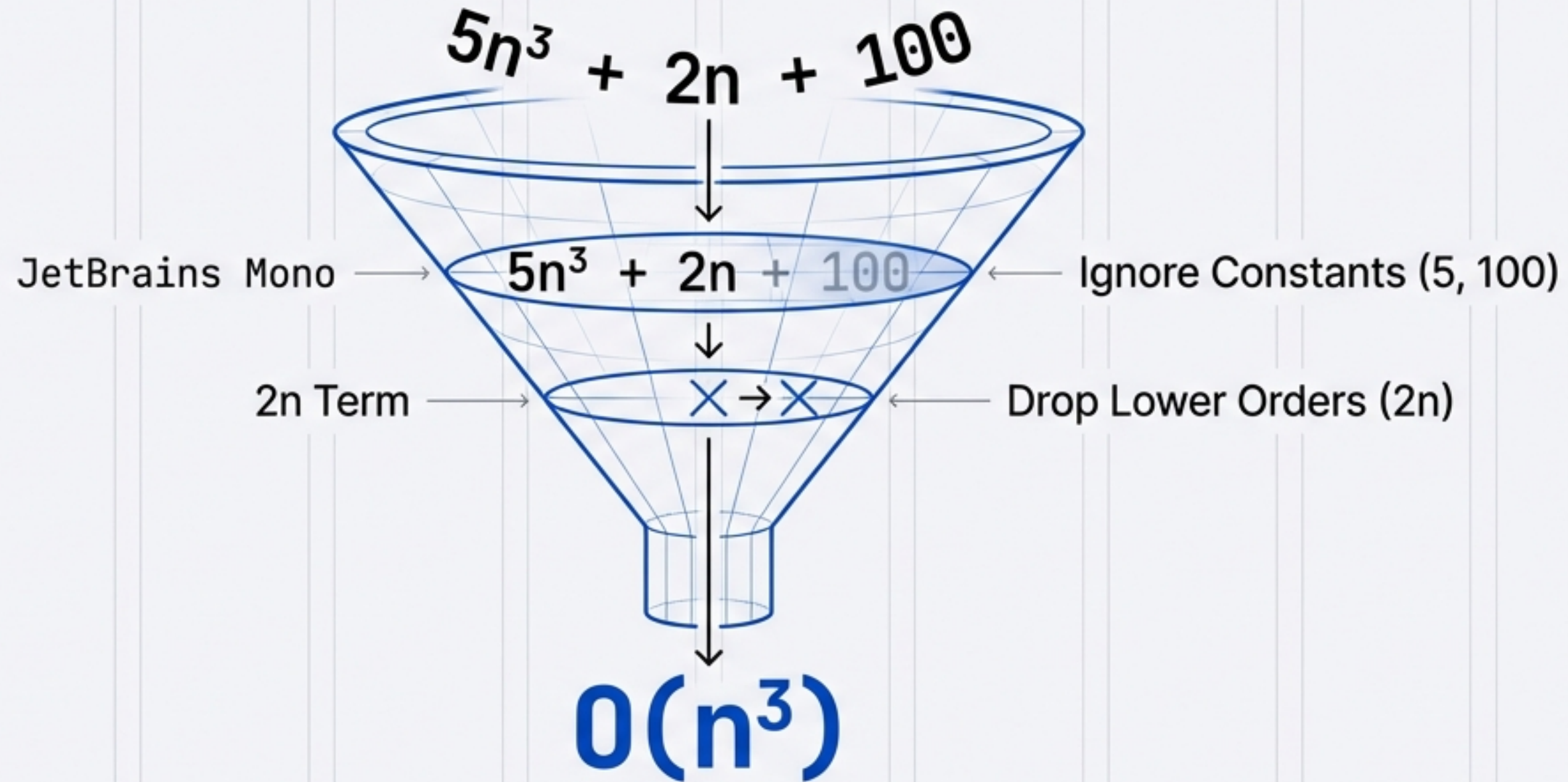


Dependent on Logic
(Input Scaling)



- Time Complexity $\neq$ Run Time. Actual speed varies by platform (LeetCode vs. CodeForces) and machine.
- We measure how the number of operations grows as Input Size (N) increases.

The Big O Filter: Ignore the Noise



KEY TAKEAWAY

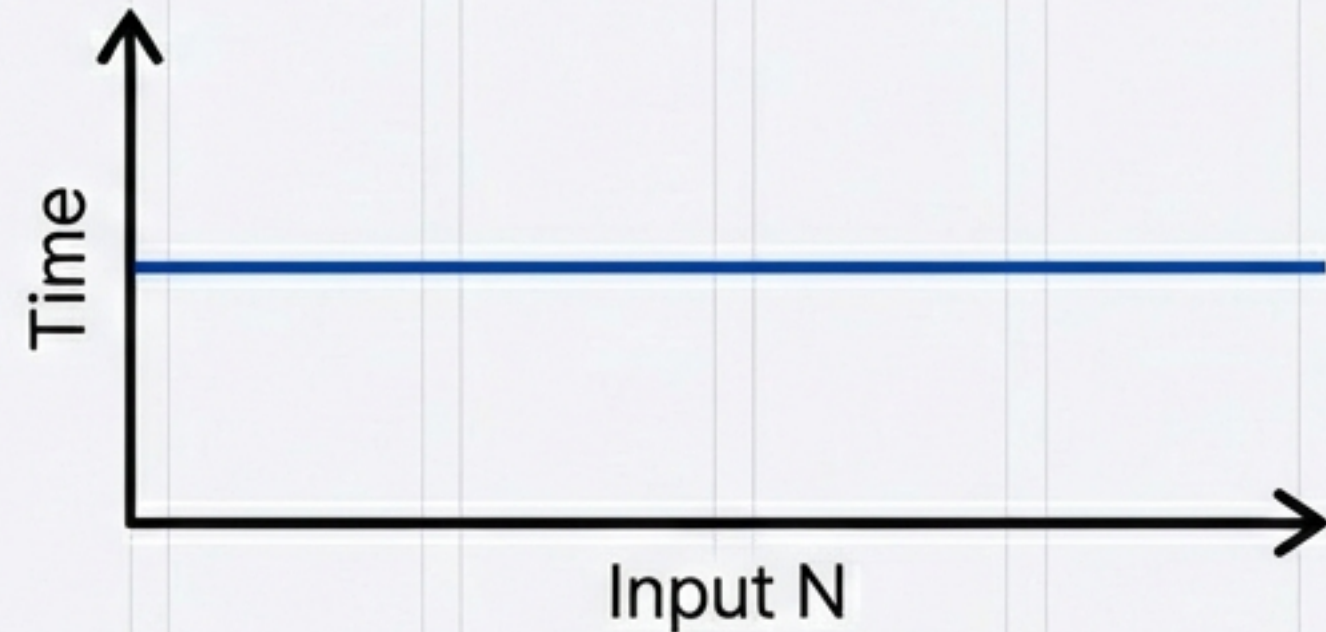
We build for the Worst Case (e.g., 1 Billion Users).

At that scale, constants and lower-order terms become irrelevant.

Rule 1: The Constant

$O(1)$

```
int x = arr[0];  
print("Hello World");
```

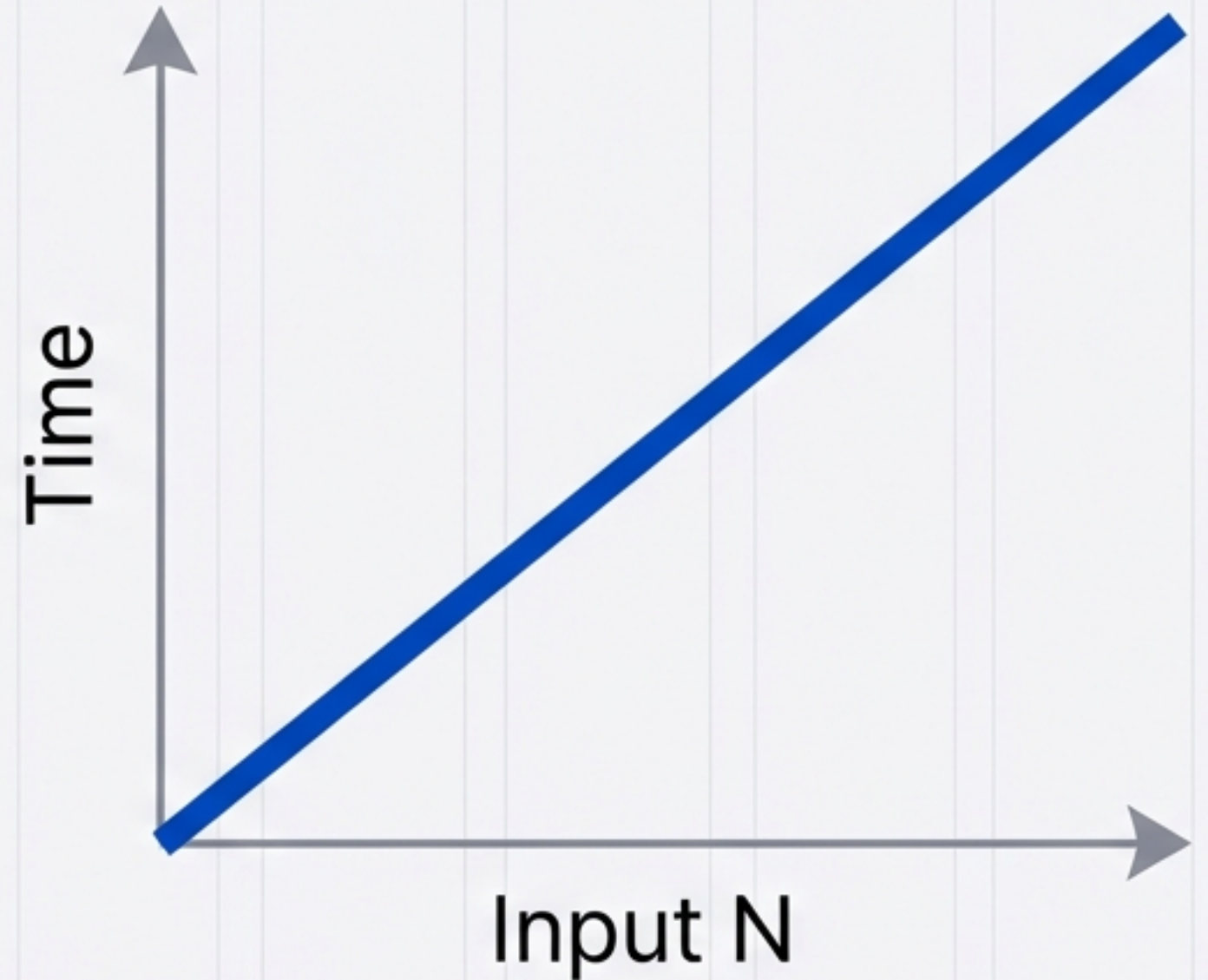


No matter if N is 10 or 10 million, the effort remains the same.

Rule 2: The Linear Loop

$O(n)$

```
for (int i = 0; i < n; i++) {  
    print(arr[i]);  
}
```



Direct Correlation: As N grows, operations grow proportionally.

Rule 3: Sequential vs. Nested

Sequential = Add



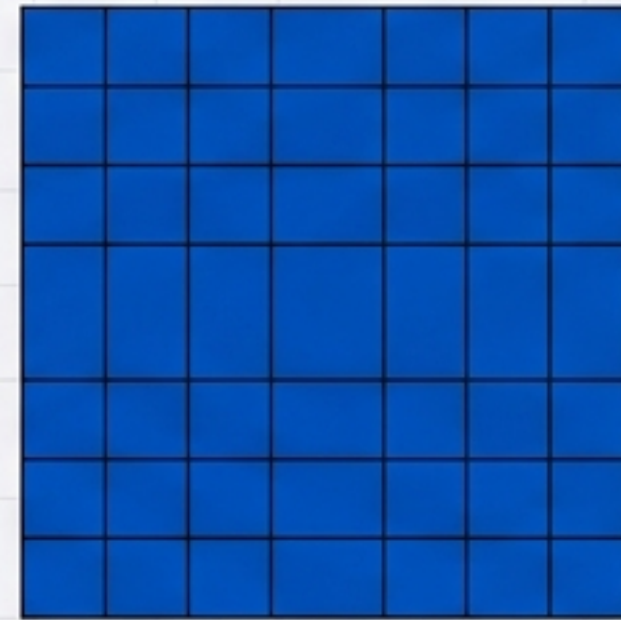
Loop A

Loop B

Loop A + Loop B = $N + N$

$O(N)$

Nested = Multiply



Loop A $\times$ Loop B = $N \times N$

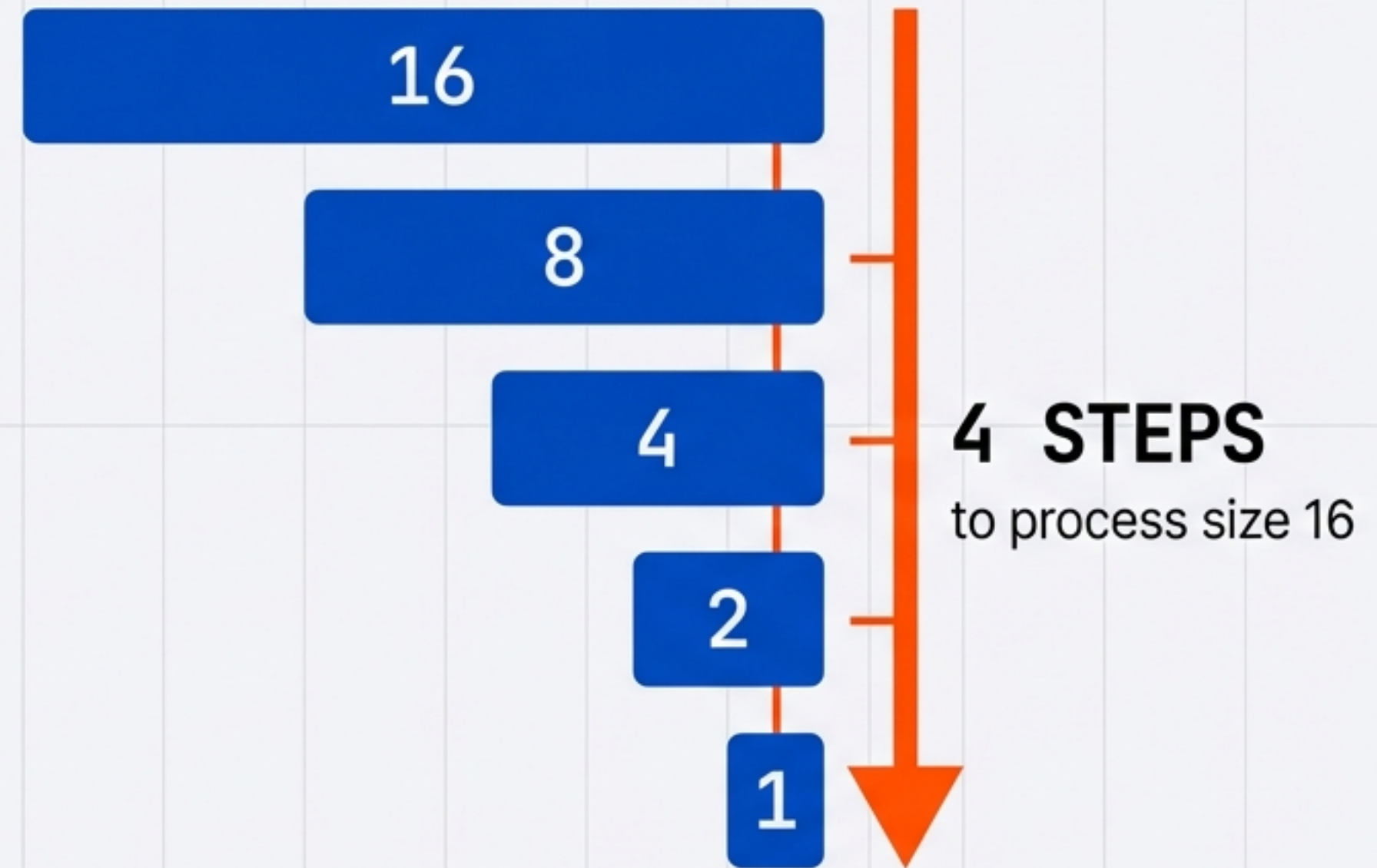
$O(N^2)$

Sequential blocks add up. Nested blocks multiply.

Rule 4: The Logarithmic Shift

$O(\log n)$

```
for (int i = 1; i < n; i = i * 2)  
// or  
binarySearch(arr, target)
```



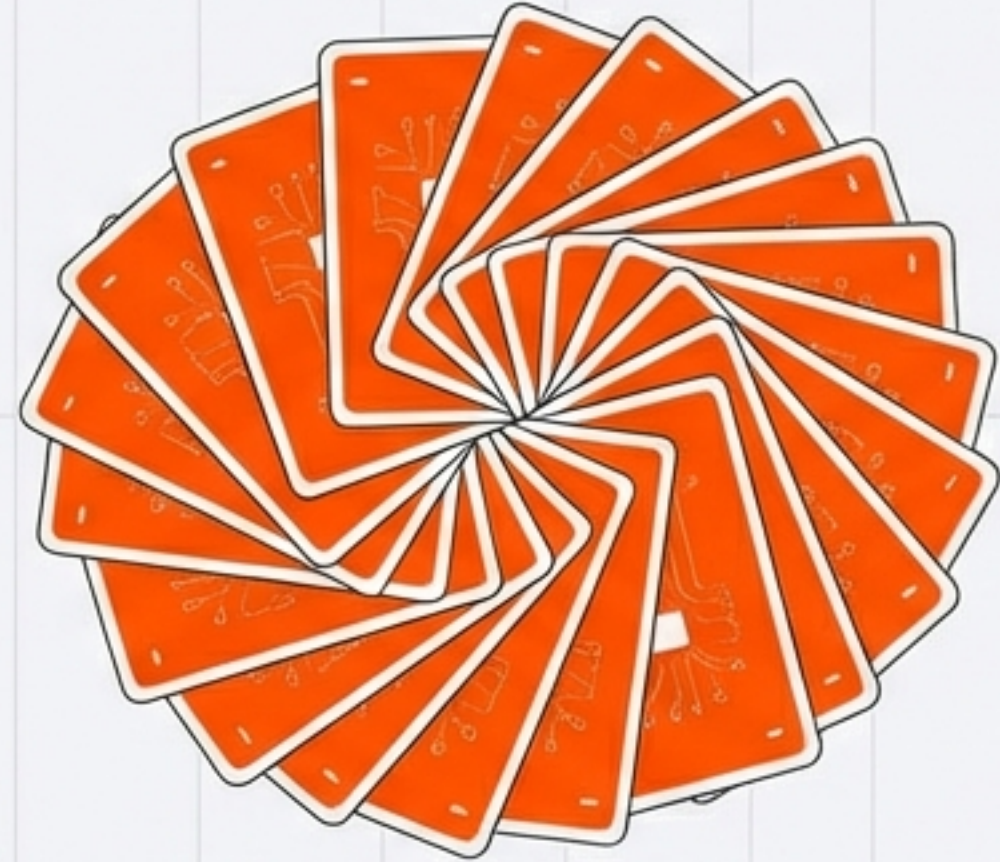
Halving or Doubling the input at every step reduces complexity drastically.

Rule 5: The Danger Zone

Exponential (Recursion) and Factorial (Permutations)



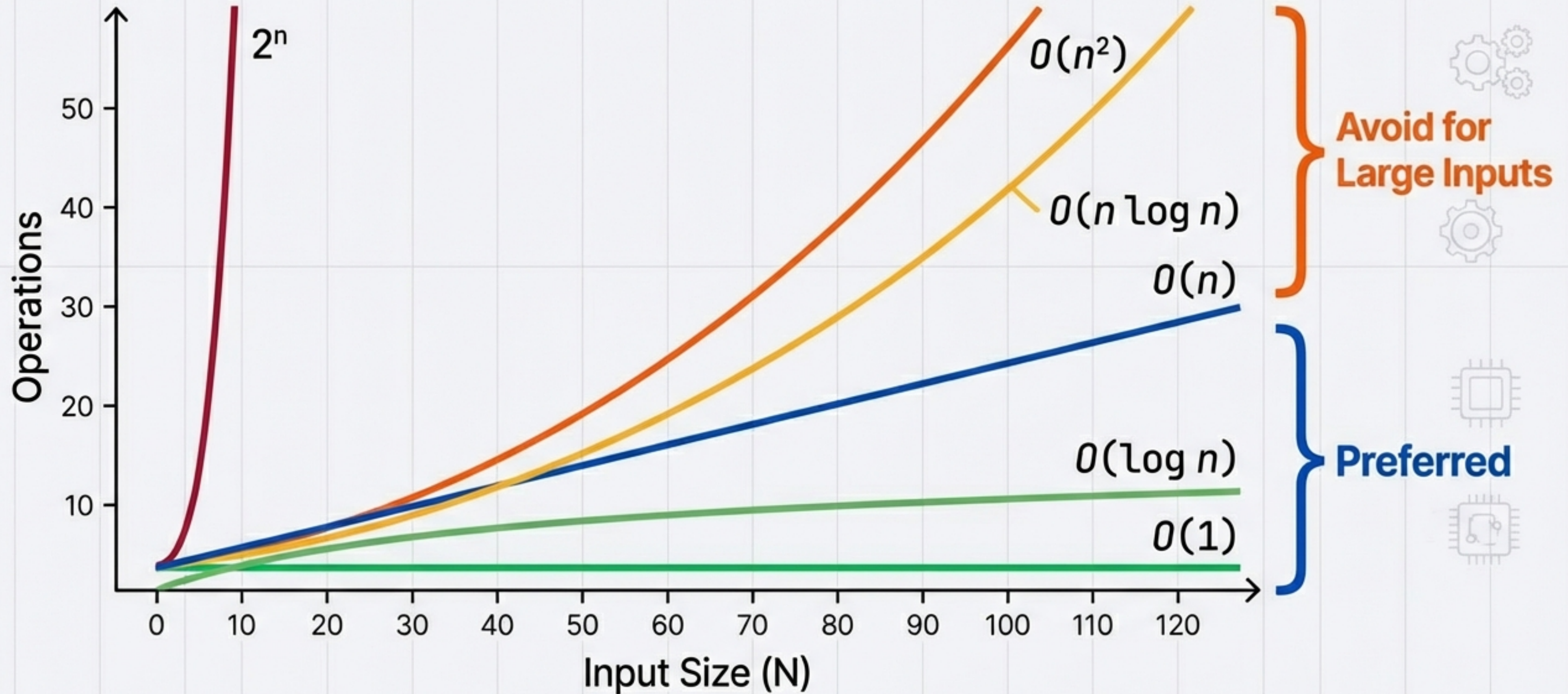
Recursion (Fibonacci)



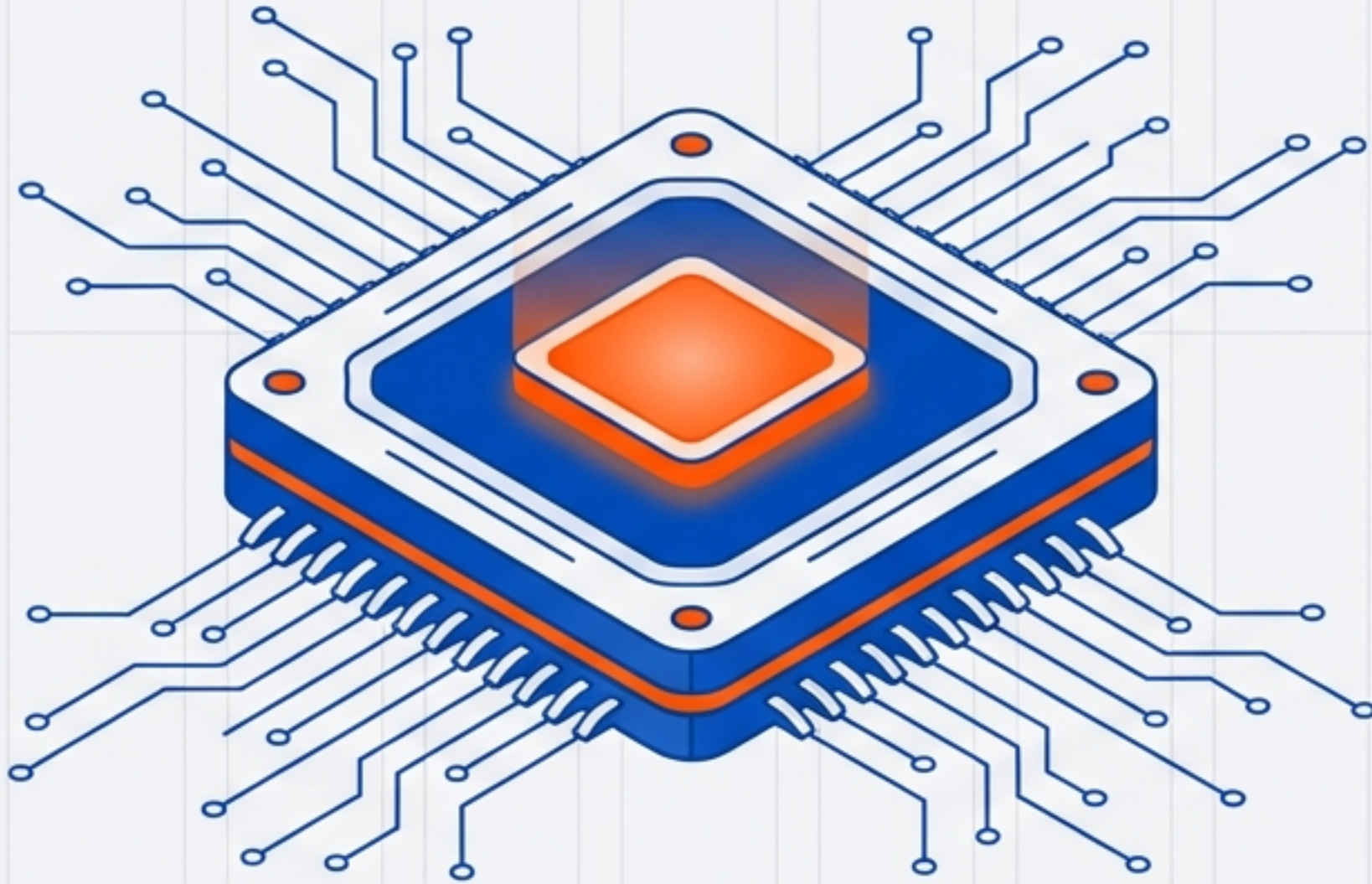
Traveling Salesman / Brute Force

Usage of these complexities creates a **vertical spike** in runtime.
Usable only for **very small N (≤ 15)**.

The Complexity Spectrum



Practical Constraints: The 1-Second Rule



\$ 10^8

- Most coding platforms (LeetCode, CodeForces) allow approximately 10^8 operations per second.
- If your Worst Case operations $> 10^8$, you will get '**Time Limit Exceeded**' (TLE).
- Use the Input Size (N) to reverse-engineer the required algorithm.

Reverse Engineering: Small Inputs

———— If $N \leq 12$ —————> ————— If $N \leq 500$ —————> ————— If $N \leq 2,000$ —————>

! $O(n!)$ or $O(2^n)$

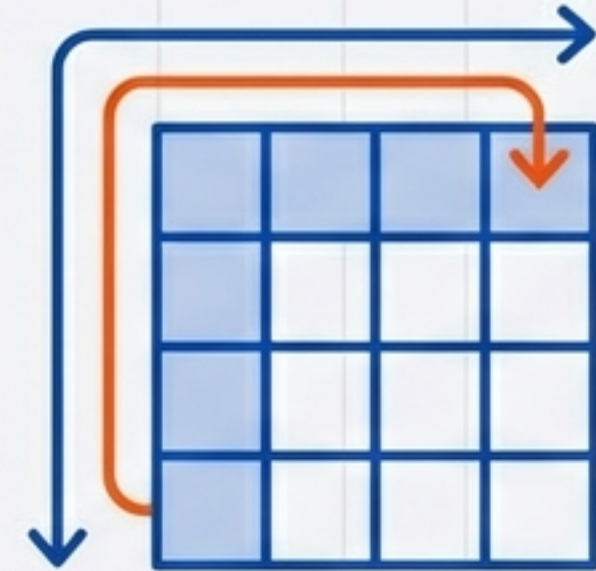
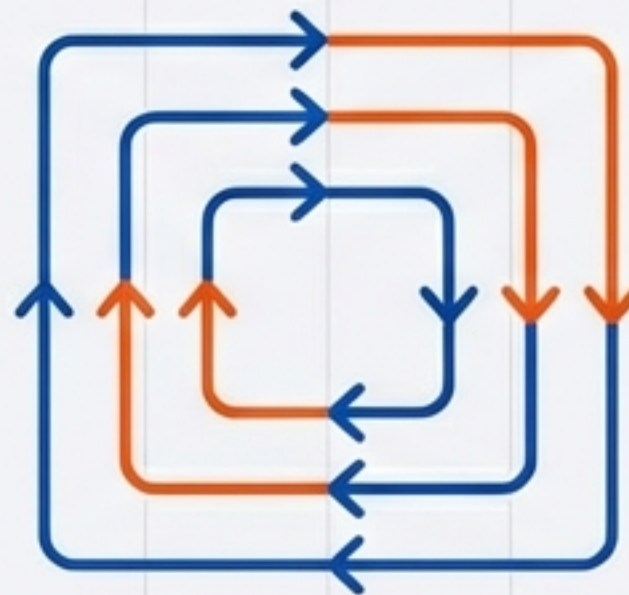
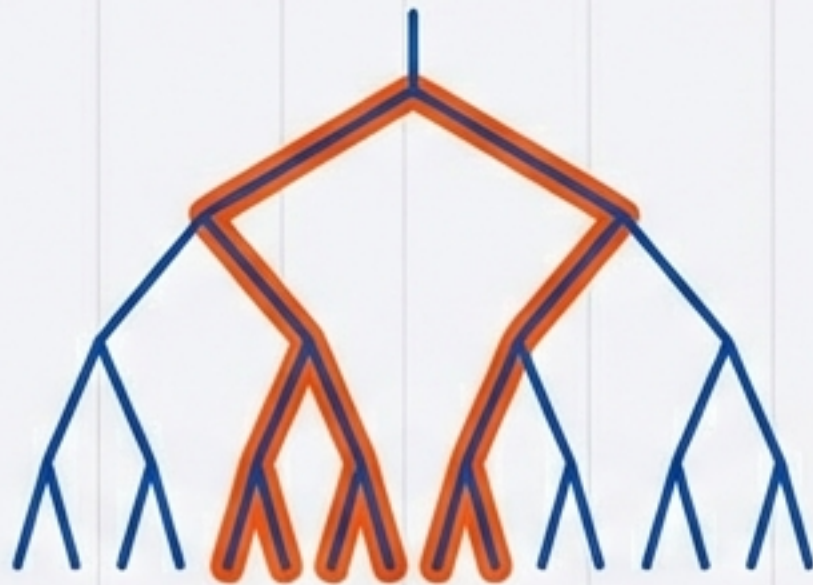
$O(n^3)$

$O(n^2)$

Hint: Recursion /
Permutations

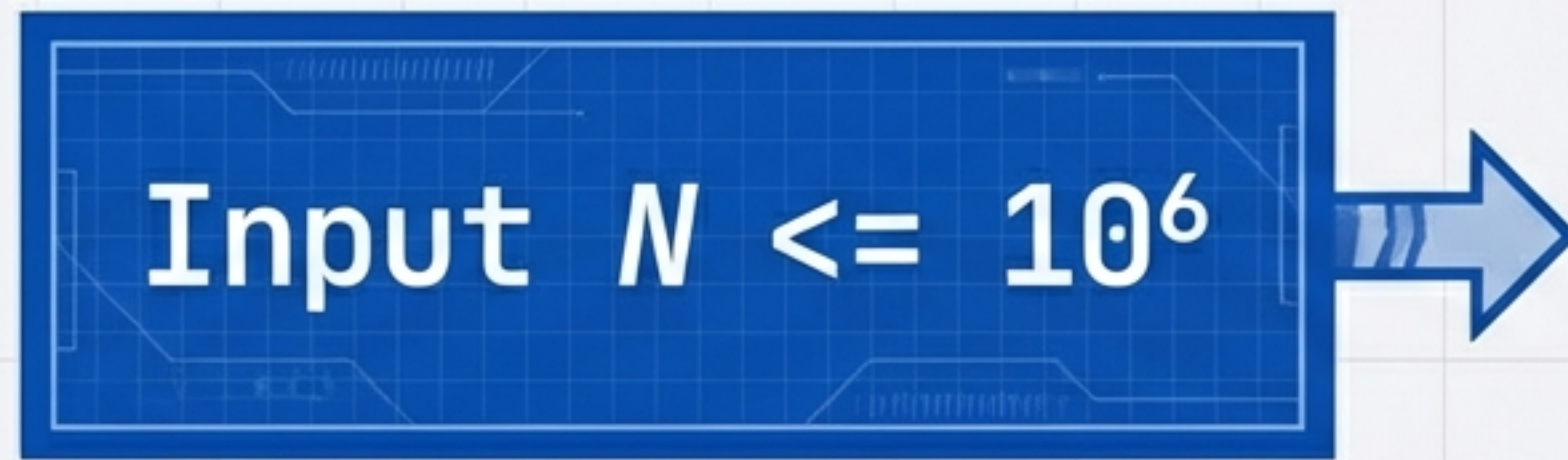
Hint: Triple Nested Loops

Hint: Double Nested Loops /
Matrix Traversal



Reverse Engineering: Medium Inputs

The Standard Interview Case (10^4 to 10^6)



Allowed Complexities:

$O(n)$
(Linear)

$O(n \log n)$
(Linearithmic)

The Logic:



Visual Hint:



Sorting



Greedy/Linear scan

Reverse Engineering: Massive Inputs

Input $N > 10^8$

At this scale, you cannot even iterate through the list once per query.

Allowed Complexities:

$O(\log n)$
(Logarithmic)

$O(1)$
(Constant)



Hint: Binary Search or Mathematical Formula.

The Master Rules Summary

Code Pattern	Complexity
Single statement	 $O(1)$
Loop running N times	 $O(n)$
Nested loops	 Multiply ($O(n^2)$)
Sequential blocks	 Add ($O(n) + O(n)$)
Halving / Doubling loop	 Logarithmic ($O(\log n)$)
Sorting	 $O(n \log n)$
Recursion with Branching	 Exponential (2^n)

From Calculation to Intuition



You don't need to write formulas for every line of code.
After 50-70 problems, you stop calculating and start "seeing" the complexity.

"Look at the loops. Check the constraints. Trust the pattern."